Merging Traffic: Benchmark of Deep Reinforcement Learning Models

Connor Slutsky
College of Eng. and Applied Sci.
University of Cincinnati
Cincinnati, USA
slutskcp@mail.uc.edu

Eli Fouts
College of Eng. and Applied Sci.
University of Cincinnati
Cincinnati, USA
foutseg@mail.uc.edu

Aarohi Shah
College of Eng. and Applied Sci.
University of Cincinnati
Cincinnati, USA
shah3ao@mail.uc.edu

Abhishek Rautela
College of Eng. and Applied Sci.
University of Cincinnati
Cincinnati, USA
rautelak@mail.uc.edu

Jace Schubert
College of Eng. and Applied Sci.
University of Cincinnati
Cincinnati, USA
schubejb@mail.uc.edu

I. Background

Our project seeks to identify a Deep Reinforcement Learning (DRL) model that is best suited to equip a car to merge onto and off of a highway. We were seeking to explore a topic in the realm of automated driving, and after some deliberation and exploration settled on specifically the act of merging onto and off of a highway. As drivers, many of us faced initial difficulty with this driving skill, so we thought it would be intriguing to explore how well DRL models would fare. We also considered it potentially challenging, such that it would be suitable for our Deep Learning class project. From our current research, we have yet to find anything that explores similar questions or tasks.

Our tool to simulate the highway environment will be the "highway-env" python package. This package is a modification of the popular "gymnasium" python package that contains tools for building environments suitable for DRL or RL model training. "highway-env" contains many pre-designed driving environments (straight highway, racetrack, parking, etc.), and ours will be a modification of the "merge-v1" environment.

II. Related Studies

The first model we reviewed was PPO+ [1], created by M. Kallel, J. Holgado-Alvarez, S. Tosatto, and C. D'Eramo. PPO+ is an implementation of Proximal Policy Optimization (PPO) [2] that improves some of the shortcomings of PPO, taking inspiration from off-policy actor-critic models such as SAC [3]. PPO is a popular [1] on-policy model created by J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov [2]. PPO utilizes actor and critic networks to find the optimal policy directly, first by using the current policy to iterate over some number of environment steps, using the critic network to compute the advantage for each action, and to compute the policy losses through mini batches of the samples collected in the previous environment to update the actor and critic networks [2]. Specifically, PPO+ aims to: 1) improve the accuracy of the critic network of PPO [1], 2) bound the action spaces [1], and 3) provide entropy regularization [1]. To address the critic accuracy, PPO+ implements a replay buffer to store past policy information to supplement the training of the critic network (instead of just using current policy information) [1]. This makes critic training off-policy, but maintains the core requirement of the actor network being on-policy [1]. For bounding the action spaces, PPO+ utilizes a hyperbolic tangent when computing the log probabilities of the policies while applying corrections for when the action value diverges at ± 1 [1]. And to address entropy regularization, PPO+ adds an entropy term in the computation of the advantage [1]. For performance, PPO+ in the MuJoCo dataset performs above average [1] in total policy return compared with PPO [2], SAC [3], and TRPO [4], and largely outperforms those same models in the DOG environment of the Deepmind Control Suite [1]. PPO+'s strengths include general strengths for on-policy models, such as stable learning and safer exploration [1], and improvements to PPO (the dominant on-policy model [1]) such as a more accurate critic, bounding the policy distribution and maximizing entropy exploration [1]. PPO+ suffers from similar weaknesses as to other on-policy models such as poor sample efficiency [1].

The second model we reviewed was Extended Off-policy Proximal Policy Optimization (ExO-PPO), developed by H. Wang and M. Yang [5]. ExO-PPO is another implementation of PPO [2], that also implements limited off-policy training to supplement the on-policy training of PPO [5]. ExO-PPO specifically seeks to balance sample efficiency (inspired by off-policy methods), and stability (one of the core benefits of PPO) [5]. ExO-PPO does this with: 1) limited off-policy training for the actor network from the last M policies stored in a replay buffer [5], 2) modifying a lower-bound function to take into account the larger range of policies [5], and 3) implementing a surrogate objective function that intends to optimize based on that modified lower-bound function with an

additional exponentially decaying edge [5]. For performance, ExO-PPO outperforms PPO, P3O [6] [5], ESPPO [7] [5], GePPO [8] on several Atari games from the OpenAI Gymnasium environment [5], and outperforms the same models + off-policy representatives SAC [3] and TD3 [9] on several MuJoCo tasks [5]. ExO-PPO's strengths include its reliable off-policy learning, and optimized gradients, but ExO-PPO also suffers a sensitivity to hyperparameters, as does PPO [5].

The third model we reviewed was Off-policy Distributional Q(λ), developed by Y. Tang, M. Rowland, R. Munos, B. Avila Pires, and W. Dabney [10]. Off-policy Distributional Q(λ) is an extension of traditional Q(λ) learning that incorporates distributional reinforcement learning [10]. Unlike traditional Q-learning methods that estimate the expected reward of an action, distributional reinforcement learning estimates the entire distribution of possible future rewards [10]. The model aims to improve multi-step off-policy learning without using importance sampling, which is commonly used to correct differences between the policy collecting experiences and the policy being trained, but can introduce high variance [10]. Specifically, Off-policy Distributional Q(λ) aims to: 1) improve learning efficiency through multi-step updates [10], 2) eliminate the need for importance sampling in off-policy learning [10], and 3) maintain stable learning by controlling the difference between the behavior policy and target policy [10]. To accomplish this, the model uses a trace parameter λ to control how much information from previous steps contributes to learning [10]. It also introduces signed measures, which allow intermediate return estimates to contain negative probability masses while maintaining a total mass of one [10]. For its deep reinforcement learning implementation, the authors combine the approach with the C51 algorithm to create Q(λ)-C51, which uses a neural network to estimate return distributions [10]. The model also introduces a mixing coefficient α to combine the greedy target policy with the behavior policy, helping improve training stability [10]. For performance, Q(λ)-C51 was evaluated on 57 Atari games and compared against C51 and distributional Retrace-C51 [10]. With $\lambda = 0.4$ and $\alpha = 0.6$, Q(λ)-C51 achieved higher interquartile mean and median normalized scores than both baseline models over the course of training [10]. The model's strengths include improved multi-step learning, the ability to reuse previously collected experiences, and avoiding the high variance associated with importance sampling [10]. However, Off-policy Distributional Q(λ) is sensitive to hyperparameter selection, particularly the value of λ, as larger values can lead to unstable learning or divergence when the behavior and target policies differ significantly [10].

The fourth model we reviewed was Reversal Q-Learning (RQL) [11], developed by A. Oberai, S. Park, and S. Levine. RQL is an off-policy reinforcement learning algorithm designed to train highly expressive, continuous flow-based policies from offline datasets without requiring unstable backpropagation through time (BPTT) or resorting to one-step policy distillation [11]. The intent of RQL is to overcome the difficulty of performing temporal difference (TD) learning over multi-step generative flow trajectories [11]. RQL achieves this with: 1) an expanded Markov Decision Process (MDP) framework that treats each intermediate flow refinement step as an individual sub-action, avoiding BPTT through the flow ODE [11], 2) a flow reversal mechanism that exploits the mathematical invertibility of deterministic flow ODEs to generate virtual on-policy rollout trajectories directly from offline dataset points [11], and 3) unbiased, zero-variance multi-step returns that collapse the artificially expanded time horizon to mitigate compounding TD error (the curse of horizon) [11]. For performance, RQL was evaluated across 50 simulated robotic manipulation and locomotion tasks, achieving the highest average return and consistently outperforming 19 competitive flow-based and diffusion-based offline RL baselines [11]. RQL's strengths include the ability to train full, multi-step expressive generative policies stably without distillation shortcuts, alongside drastically reduced value-estimation variance through exact ODE inversion [11]. However, RQL's primary weaknesses stem from the computational latency of integrating multi-step ODEs at inference time and its reliance on deterministic flow formulations [11].

The fifth model we reviewed was SimBa [12], which was created by H. Lee, D. Hwang, et al. [12]. Compared to the other models we reviewed, SimBa is not a learning algorithm and it is instead a network architecture that can be plugged into existing DRL algorithms such as SAC [3] and PPO [2], which it was tested on [12]. SimBa is designed to lead the network towards simpler solutions, allowing it to become bigger while minimizing overfitting [12]. It does this using three main components [12]. Firstly, SimBa normalizes the inputs by keeping note of the running mean and variance for each input, which puts those inputs on a similar scale to make it less likely that larger inputs overpower the smaller ones [12]. Secondly, SimBa uses residual blocks where each layer takes the input and adds a minor change, which leads the network towards simpler solutions since it can pass inputs mostly unchanged if there is nothing useful to add [12]. Lastly, SimBa uses layer normalization inside each block and before the output, keeping the values inside the network steady and making the training more stable [12]. For performance, SimBa was tested with SAC on 51 tasks from the DeepMind Control Suite, MyoSuite and HumanoidBench, where it matched or outperformed other state-of-the-art off-policy models while using less compute [12]. On the hardest DeepMind Control tasks, adding SimBa increased the average return of SAC by about 570, showing that changing the network architecture can improve performance without changing the learning algorithm [12]. SimBa was also tested with PPO on Craftax, where it allowed the agent to progress through the game faster than with a standard network [12]. SimBa's strengths include how easily it can be added to existing DRL algorithms and how it allows the network to become bigger while minimizing overfitting [12]. However, its improvements were smaller on tasks with fewer inputs, and making the actor network bigger showed little to no benefit [12]. SimBa was mostly tested on continuous control tasks, and the reviewed experiments did

not include driving tasks such as highway merging, which is something our project can explore.

III. Preliminary Benchmarking

To get an early baseline, we built a test version of our environment in our "profile_test" folder and benchmarked three classic models from Stable-Baselines3: DQN, PPO [2], and A2C. Every episode generates a random road where the car starts on a ramp, merges into traffic made up of normal, slow, fast, and reckless drivers, and then has to take an exit 250 to 500 meters down the road. All three models used discrete actions, the same reward function, and the same network size, and each trained for about 300,000 steps. We then scored them on the same 30 unseen roads, along with a rule-based driver that follows the same rules as the traffic so that we had something to compare against. The rule-based driver made the exit 70% of the time and crashed 20% of the time. DQN and PPO both made the exit 47% of the time and crashed the other 53%, while A2C only made the exit 10% of the time. None of the models have beaten the rule-based driver yet, and since we only trained one seed per model, we would not read too much into the differences between them. From here, we plan to move this setup into our main environment with the full 1 km road and add the 2026 models, starting with PPO+ [1] and ExO-PPO [5] since both build on PPO and should hopefully be easier to plug in. We also plan to run at least 5 seeds per model with 50 or more evaluation episodes each, and report the interquartile mean with bootstrapped confidence intervals [13] instead of a single average. We would focus on collision rate and time-to-exit, since reward alone can favor a model just because of how we weighted it. We still need to pick one action space for the whole benchmark, because our tests use discrete actions while many recent models were built for continuous control.

References

[1] M. Kallel, J. Holgado-Alvarez, S. Tosatto, and C. D'Eramo, "PPO+: Enhancing proximal policy optimization," Reinforcement Learning Journal, 2026. [Online]. Available: https://rlj.cs.umass.edu/2026/papers/Paper8.pdf

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," Aug. 2017, arXiv:1707.06347. [Online]. Available: https://arxiv.org/pdf/1707.06347

[3] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor," Berkeley Artificial Intelligence Research, University of California, Berkeley, Aug. 2018, arXiv:1801.01290. [Online]. Available: https://arxiv.org/pdf/1801.01290

[4] J. Schulman, S. Levine, P. Moritz, M. Jordan, and P. Abbeel, "Trust Region Policy Optimization," Proceedings of the 31st International Conference on Machine Learning, Apr. 2017, arXiv:1502.05477. [Online]. Available: https://arxiv.org/pdf/1502.05477

[5] H. Wang and M. Yang, "ExO-PPO: an Extended Off-policy Proximal Policy Optimization Algorithm," School of Aeronautics and Astronautics Organization, Feb. 2026, arXiv:2602.09726. [Online]. Available: https://arxiv.org/pdf/2602.09726

[6] X. Chen, et al., "The sufficiency of off-policyness and soft clipping: PPO is still insufficient according to an off-policy measure," Proceedings of the AAAI Conference on Artificial Intelligence 37(6), 7078–7086, 2023. https://doi.org/10.1609/aaai.v37i6.25864

[7] M. Sun et al. "You May Not Need Ratio Clipping in PPO", 2022

[8] J. Queeney, I. C. Paschalidis, C. Cassandras, "Generalized Proximal Policy Optimization with Sample Reuse," 35th Conference on Neural Information Processing Systems, Oct. 2021, arXiv:2111.00072. [Online]. Available: https://arxiv.org/pdf/2111.00072

[9] S. Fujimoto, H. van Hoof, D. Meger, "Addressing Function Approximation Error in Actor-Critic Methods," Proceedings of the 35th International Conference on Machine Learning, Oct. 2018, arXiv:1802.09477. [Online]. Available: https://arxiv.org/pdf/1802.09477

[10] Y. Tang, M. Rowland, R. Munos, B. Avila Pires, and W. Dabney, "Off-policy Distributional Q(λ): Distributional RL Without Importance Sampling," 29th International Conference of AISTATS, vol. 300, 2026. [Online]. Available: https://raw.githubusercontent.com/mlresearch/v300/main/assets/tang26a/tang26a.pdf

[11] A. Oberai, S. Park, S. Levine, "Reversal Q-Learning," University of California, Berkeley, June 2026, arXiv:2606.17551. [Online]. Available: https://arxiv.org/pdf/2606.17551

[12] H. Lee, et al. "SIMBA: SIMPLICITY BIAS FOR SCALING UP PARAMETERS IN DEEP REINFORCEMENT LEARNING," ICLR, 2025, arXiv:2410.09754. [Online]. Available: https://arxiv.org/pdf/2410.09754

[13] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, "Deep Reinforcement Learning at the Edge of the Statistical Precipice," 35th Conference on Neural Information Processing Systems, 2021, arXiv:2108.13264. [Online]. Available: https://arxiv.org/pdf/2108.13264